Zcash Name Service

A Deterministic, Attested Name Registry on Zcash

Julian Abraham julian@zcash.me
James Joseph james@zcash.me
Lana Ivina lana@circuitlabs.io
github.com/zcashme

November 13, 2026

License: MIT

Discussion: github.com/zcashme/zns-whitepaper

Abstract

Zcash payments use encoded addresses, but Zcash consensus does not assign human-readable names to them. Existing on-chain naming systems cannot be ported to Zcash: transparent registries such as Namecoin publish the namespace in the clear, leaking the address graph Zcash exists to hide; contract-based registries such as ENS require general-purpose on-chain state that Zcash consensus does not provide. The Zcash Name Service (ZNS) maps each Zcash name to a Unified Address over the ordinary Zcash transaction log: each binding is recorded in the memo of an Ironwood action, decryptable with the Name Note account's published full viewing key, and anchored by an Ironwood note commitment derived from the same tuple. Zcash consensus supplies canonical order and finality; a deterministic reducer derives the registry; the bound Unified Address inherits Ironwood shielding. Authorization is delivered by an attested Mint; a future off-chain zero-knowledge name circuit would replace that assumption with a proof.

Status. This document is a work in progress. Sections explicitly marked Open or Provisional are not interoperability requirements. Sections marked Fixed state normative requirements backed by the cited assumptions. An implementation cannot infer a protocol rule from an unresolved item. The key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are to be interpreted as described by BCP 14 only when they appear in uppercase [6].

1 Motivation

Zcash addresses are long encoded strings. A naming system maps these strings to human-readable labels, but this requires enforcing rules about which names are claimed and who controls them. Zcash consensus tracks unspent coins. Lacking a virtual machine to run logic and a global state tree to store variables, it cannot evaluate a naming policy. To bypass this limitation, ZNS separates the execution of the naming policy from the permanent public record. A single party, the Mint, executes the policy off-chain. When a request passes, the Mint writes the resulting name binding into the 512-byte memo field of a shielded Zcash transaction. Zcash consensus seals the transaction into a block, locking the binding into a permanent, chronological order. A Resolver looking up a name downloads the Zcash block headers, uses a published viewing key to decrypt the Mint's memos, and reads the bindings in the exact order consensus finalized them. The Mint issues names, but the accumulated work of the Zcash blockchain prevents it from altering or erasing previously recorded bindings.

2 Architecture

To guarantee the globally unique mapping of human-readable names to Zcash addresses, a naming system requires an agreed-upon history of requests to register, modify, or release names, a strict set of rules, and a resulting registry of name bindings. ZNS treats this registry as derived state by using Zcash exclusively for the history of these requests, executing rules off-chain, and binding their accepted results on-chain.

2.1 System Components

The protocol relies on three distinct components:

- **The Mint.** The authoritative off-chain program that evaluates the naming policy. It runs inside a Trusted Execution Environment (TEE). The Mint spending key is generated in the enclave, and the Mint code and pricing policy execute there. Remote attestation binds that key and those programs to an approved enclave measurement. It uses its own Zcash node to read chain state independently.
- **The Name Note.** An Ironwood output used to record a name transition on-chain.
- **The Resolver.** A permissionless client that reads the canonical sequence of Zcash transactions, verifies Name Notes and deterministically derives

registry state.

2.2 Independence of Facts

The security of the derived state relies on six distinct properties. The Resolver verifies publicly reconstructible properties directly and relies on Mint attestation for policy checks that are not publicly reconstructible.

- **Zcash validity:** The transaction satisfies Zcash consensus.
- **Mint authorization:** The recognized Mint spending key authorized the transaction.
- **TEE attestation:** The Mint spending key was generated in the approved enclave, which executes the Mint code and pricing policy.
- **Commitment verification:** The transition tuple encoded in a Name Note matches its Ironwood note commitment.
- **User authorization:** The attested Mint enforces the authorization procedure defined in Section 6.
- **State derivation:** Resolvers reading the canonical Zcash chain under identical protocol parameters derive the same registry state.

3 User requests

A Request asks the Mint to evaluate a name change. A Respond returns an OTP for a pending Request. Neither is a Name Note and neither binds a transition to a note commitment.

3.1 Request encoding

User-to-Mint memos are a 512-byte field. A parser removes the maximal trailing sequence of zero bytes. The remaining bytes MUST be UTF-8. The colon byte (`0x3a`) separates fields. The first field MUST be the exact ASCII bytes `ZNS`. Extra fields, missing fields, and empty required fields are rejected.

The name field MUST contain between 1 and 63 bytes, each of which is ASCII `a`–`z` or `0`–`9`.

A Request has one of the following forms:

```
ZNS:claim:<term>:<name>:<ua>
ZNS:claim:<code>:<term>:<name>:<ua>
ZNS:update:<term>:<name>:<ua>
```

```
ZNS:release:<name>:<ua>
```

The `<term>` field of a `claim` is either the exact ASCII bytes `forever` or `<N>y`, where `<N>` is a canonical decimal integer from 1 to 99 with no leading zeroes. `forever` requests a registration without fixed expiration. The `<term>` field of an `update` is one of: the exact ASCII bytes `none`, which carries the current `expires_at` forward unchanged; `<N>y` under the same spelling rules, which requests an extension of `<N>` years; or the exact ASCII bytes `forever`, which requests an upgrade of a fixed-term registration to a registration without fixed expiration. An `update` MUST NOT use `forever` against a registration whose `expires_at` is `none`: the upgrade applies only to a fixed-term registration. A `claim` MUST NOT use `none`: it says `forever`. No field of any memo is empty.

An optional leading `<code>` field may appear only on a `claim`. It is exactly six ASCII decimal digits, `0`–`9`, including leading zeroes. A parser that sees a leading field that parses as `forever` or `<N>y` MUST treat it as the term (the three-field claim form). Any other leading field that is exactly six decimal digits is the access code. A leading field that is neither a valid term nor a valid access code is rejected. Update and release Requests MUST NOT carry an access code. The access code is not an OTP and is never present on a Respond.

A Respond has one of the following forms:

```
ZNS:otp:<otp>:<name>:<verb>:<ua>
```

The Relay and the Respond have the same form and differ only in sender: the Mint sends the Relay to the controller, and the controller sends the Respond back to the Mint. The controller constructs nothing; the memo is copied back.

A Request or Respond MUST include a non-empty Unified Address. A `claim` is only a Request. Request and Respond memos do not encode `expires_at` or `prev`.

An `update` or `release` Request is the first step of the authorization procedure specified in Section 6. A Respond is the third step: the controller sending the OTP back to the Mint. The `<otp>` field is exactly six ASCII decimal digits, including leading zeroes.

A Request parser MUST reject any memo that does not match one of the forms above, including the `otp` form. A Mint-side parser accepts the `otp` form as a Respond.

4 Name Notes

A Name Note is an Ironwood output whose memo records a name transition by encoding the type of name change, the name, the associated Unified Address when applicable, the expiration value, and a predecessor reference. Its `rseed` is derived from that transition, so the note commitment binds the transition the memo records.

4.1 Memo encoding

The Name Note memo is a 512-byte field. A parser removes its maximal trailing sequence of zero bytes. The remaining bytes MUST be UTF-8. The colon byte (`0x3a`) separates fields. The first field MUST be the exact ASCII bytes `ZNS`. Extra fields, missing fields, and empty required fields are rejected.

The name field MUST contain between 1 and 63 bytes, each of which is ASCII `a`–`z` or `0`–`9`.

The remaining bytes encode one transition in one of the following formats:

```
ZNS:claim:<name>:<ua>:<expires_at>:<prev>
ZNS:update:<name>:<ua>:<expires_at>:<prev>
ZNS:release:<name>:<ua>:none:<prev>
```

The `<expires_at>` field is either the canonical ASCII decimal encoding of a Unix timestamp in whole seconds or the exact ASCII bytes `none`. The value `none` indicates that the registration has no fixed expiration. A `release` MUST encode `none` as its `expires_at` value and the Unified Address of the binding being released as its `ua` value.

The `prev` field is the 64-character lowercase hexadecimal encoding of the predecessor Name Note's `cmx`: the 32-byte canonical little-endian encoding of its extracted note commitment, as recorded in the on-chain action that created it. A `claim` has no predecessor and uses 64 zeroes.

A Name Note parser MUST reject a memo that does not match one of these encodings.

4.2 Transition representation

To derive the note's `rseed`, interpret the encoded transition as the tuple

$$\sigma = (\alpha, n, u, e, p)$$

where:

- α: The exact ASCII bytes `claim`, `update`, or `release`.
- n: The raw bytes of the name field.
- u: The raw bytes of the `ua` field.
- e: The raw bytes of the `expires_at` field.
- p: The 32 raw bytes decoded from `prev`.

Define the fields f of the transition as

$$f = \alpha \parallel \texttt{0x3a} \parallel n \parallel \texttt{0x3a} \parallel u \parallel \texttt{0x3a} \parallel e.$$

These are the memo bytes that follow the leading `ZNS:` and precede the colon before `prev`.

4.3 Commitment inputs

A Name Note is a bound note: the sender does not sample its `rseed` at random but derives it from the transition.

Define the binding domain D as the 16 ASCII bytes `ZNS___Name_Notes`, and the record kind κ as the single byte `0x01`. Let ℓ be the byte length of f. Then

$$\mathsf{rseed}_\sigma = \mathsf{BLAKE2b{-}256}\big(D,\ \kappa \parallel \mathsf{I2LEOSP}_{64}(\ell) \parallel f \parallel p\big),$$

where $\mathsf{BLAKE2b{-}256}(D, x)$ is unkeyed BLAKE2b with a 32-byte output, the 16-byte personalization D, and input x. $\mathsf{I2LEOSP}_{64}(\ell)$ is the eight-byte unsigned little-endian encoding of ℓ. The final p is exactly 32 raw bytes and has no length prefix. The length prefix separates f from p, so distinct transitions give distinct hash inputs.

Because e is included in f, the `rseed` cryptographically binds the expiration value recorded in the Name Note. Because p is included, it binds the predecessor.

The remaining commitment inputs follow from rseed_σ exactly as for any Ironwood note, whose note plaintext lead byte is `0x03` [4]. Let $\underline{\rho}$ be the 32-byte encoding of the note's ρ, which is the nullifier revealed by the same action. Let $\mathsf{g}\star_\mathsf{d}$ and $\mathsf{pk}\star_\mathsf{d}$ be the 32-byte encodings of the recipient's diversified base and transmission key, and v the note value. Then

$$\psi_\sigma = \mathsf{ToBase}\big(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{rseed}_\sigma}(\texttt{0x09} \parallel \underline{\rho})\big),$$

$$\mathsf{pre_rcm} = \texttt{0x0B} \parallel \mathsf{g}\star_{\mathsf{d}} \parallel \mathsf{pk}\star_{\mathsf{d}} \parallel \mathsf{I2LEOSP}_{64}(\mathsf{v}) \parallel \underline{\rho} \parallel \mathsf{I2LEOSP}_{256}(\psi_\sigma),$$

$$\mathsf{rcm}_\sigma = \mathsf{ToScalar}\big(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{rseed}_\sigma}(\mathsf{pre_rcm})\big).$$

$\mathsf{PRF}^{\mathsf{expand}}$, ToScalar, ToBase, and the field types are those defined for Ironwood by the Zcash Protocol Specification and ZIP 2005 [1, 3, 4]. The value rcm_σ is the Pallas scalar-field element used as the Ironwood note-commitment randomness. The value ψ_σ is the Pallas base-field element supplied to the Ironwood note commitment construction. The canonical encoding of either field element is its 32-byte little-endian field representation.

The note plaintext carries rseed_σ as usual. A Name Note is otherwise an ordinary Ironwood note: a receiver that follows ZIP 212 and ZIP 2005 decrypts it and recomputes its commitment without ZNS-specific rules [2, 4].

Changing the binding domain, the record kind, field order, the field separator, the length width, byte order, hash parameters, or the predecessor encoding changes the protocol version and requires new test vectors.

4.4 Commitment verification

A verifier derives rseed_σ from the encoded transition, derives ψ_σ and rcm_σ from it, combines them with the remaining note components using the standard construction [1, 3], and reconstructs the note commitment. The verifier needs the recipient address, the note value, and the action's nullifier and `cmx`. It needs no decryption key.

The Name Note passes commitment verification only if the reconstructed value equals the `cmx` recorded in the on-chain action.

This equality establishes only that the transition encoded in the memo is cryptographically bound to that output. It does not establish Mint authorization, user authorization, or the current registry state.

4.5 Commitment test vector

For a `claim` transition with `name=alice`, `expires_at=none`, and a 32-byte zero predecessor. Key material is derived from the all-zero 32-byte seed under unified account $\texttt{m/32}'\texttt{/133}'\texttt{/1}'$, diversifier index 0, external scope, mainnet:

Value	Canonical little-endian bytes rendered as hexadecimal
g_d	de4338f2ab9fd8300a3a1c20dd690ce27026c6001c295d7c641a067ce809b11e
pk_d	6df609f5710f3b5deecd4ee4b8f0173b44af6cf8918ac00269526031ba628996
rseed_σ	5bcb6b8f30d7f8cd75f253b738c6fb034841e31c70d137373f3b36b0c595e56d
ψ_σ	b174aa83656d538b12d561e9048352bbd8bf1b66fe8d396977b4b597b962f70d
rcm_σ	19cb8bab986250c69207bb4cc005f9944f913c635d543ed3323fcd5f1b7b8d1e

ua: u1897y9pzw3zk6n9twtzu2z5kpkzw3hms2c54fpyv8lnr79m73ta
zljkk3veaxrtwncp66lf45p3f274xy2amqckx0sraje4v835yw8q0q

The fields f are 123 bytes. rseed_σ is a byte string and is rendered as it is. Set the value to zero and ρ to 32 bytes of 0x33. The resulting cmx is

617b189869560cdd3928456693c1d4060752f18173425294141641e4b05d0f1a

An implementation that produces another value is non-conforming.

The complete memo encoding of this transition is:

ZNS:claim:alice:u1897y9pzw3zk6n9twtzu2z5kpkzw3hms2c54fpyv8lnr79m73ta
zljkk3veaxrtwncp66lf45p3f274xy2amqckx0sraje4v835yw8q0q:none:
0000000000000000000000000000000000000000000000000000000000000000

5 The Mint

The Mint is the protocol's single registrar. A Name Note establishes that a transition is bound to an Ironwood output, but whether the transition is allowed depends on the current registry state and active naming policy. The Mint reads ordered Request and Respond memos on Zcash, encoded as specified in Section 3, derives that state, applies the policy, and for each accepted Request or Respond spends a Mint-controlled note to create the successor Name Note.

5.1 Authority bound to policy

The Mint spending key provides the protocol's Mint authorization. The key can create a transition that appears to have been approved by the Mint without applying the naming policy. The key must therefore be generated in, and usable only by, the program that evaluates the policy.

ZNS runs the Mint inside a Trusted Execution Environment (TEE). The Mint spending key is generated in the enclave. The Mint code and pricing

policy execute in that enclave. Remote attestation binds the key and those programs to an approved enclave measurement. This shifts trust in policy execution and key custody from the Mint operator to the TEE attestation system and its hardware root of trust.

5.2 Attestation constraints

The Resolver recognizes Mint authorization only if the attestation establishes all of the following:

1. The enclave code measurement is approved by the current ZNS deployment.
2. The enclave generated the Mint spending key and is the only party that can use it.
3. The enclave runs the applicable ZNS protocol version.
4. The enclave targets the applicable Zcash network.
5. The enclave implements the required protected persistent-state mechanism for replay prevention.
6. The enclave obtains canonical-chain Median Time Past from the approved Zcash node interface and uses it as the authoritative time source for protocol-defined time checks.

The Resolver rejects the Mint authorization if any condition fails or if the TEE is not running in an approved production security configuration.

Approved enclave measurements, attestation requirements, and the approved source of canonical-chain Median Time Past are deployment-specific. The exact attestation format and verification procedure depend on the selected TEE platform and remain Open until the deployment is fixed.

5.3 Claim eligibility

Some names may be designated as protected to reduce impersonation, misleading claims of affiliation, and opportunistic registration of names associated with existing identities or projects. Protection applies per name: the protected-name policy either fixes the time at which a name's protection lifts or keeps the name protected indefinitely.

The Mint consults the attested protected-name policy to classify each claim name as open or protected at the canonical-chain MTP of the claim. A name whose protection has lifted is open. An open name follows the ordinary claim path, and any access code it carries is ignored. A protected name is accepted only if the claim Request carries the matching access code in the leading slot

specified in Section 3. A missing or incorrect code is rejected; the payment is retained by the Mint as for any other rejected claim. The Mint evaluates claims against its most recently loaded copy of the policy and does not wait on the policy data source.

Redemption is derived from registry state: once a protected name has an accepted live Name Note, further claims for that name fail under the ordinary namespace rules. The Mint does not mark codes spent in an external store.

Access-code verification is Mint policy under attestation and is not independently reconstructed by the Resolver. Once a protected name is validly claimed, it is governed by the same transition and authorization rules as any other name.

5.4 Request evaluation

The Mint evaluates each request against the registry state it derives from the canonical Zcash chain. It must use its own Zcash node and its own replay of the namespace history rather than rely on a third-party state oracle. The Mint obtains canonical-chain Median Time Past from that node for protocol-defined time checks.

The Mint evaluates each request against the following requirements:

- **Namespace state.** The requested change must be permitted by the current state of the name.
- **Claim eligibility.** A `claim` must satisfy the protected-name rules in Claim eligibility above, including any required access code.
- **Payment.** The memo must satisfy the price and payment requirements defined by the active pricing policy. For a `claim`, that payment is the Request. For an `update` or controller-requested `release`, that payment is the Respond specified in Section 6.
- **Request validity.** The Request or Respond must satisfy Section 3. The name, Unified Address when applicable, requested registration term or extension when applicable, and resulting encoded transition must satisfy the protocol's validity rules.
- **Lifecycle policy.** The request must satisfy the active registration, expiration, renewal, and liveness rules defined in Section 5.5.
- **User authorization.** An `update` or controller-requested `release` must complete the authorization procedure defined in Section 6. A lifecycle `release` created by the Mint to enforce expiration or liveness does not require user authorization.

If all applicable requirements are satisfied, the Mint accepts the request and creates the resulting Name Note.

5.5 Name lifecycle

ZNS uses canonical-chain Median Time Past (MTP) rather than block counts or local system time for protocol-defined lifecycle periods. MTP is derived from Zcash block timestamps and has one-second granularity. It is chain-derived time rather than exact wall-clock time.

5.5.1 Initial claims

A `claim` request specifies the name, the Unified Address, and either a requested registration term or no fixed expiration. The user does not supply an absolute `expires_at` value. A requested registration term MUST NOT exceed 99 years.

For a fixed-term registration, the Mint computes `expires_at` by adding the requested registration term to the canonical-chain MTP of the block containing the accepted claim request.

The Mint determines `expires_at` from that block MTP and the requested registration term, subject to the active registration policy, and records the resulting value in the Name Note.

For a registration without a fixed expiration, `expires_at` is the exact ASCII value `none`.

Only the resulting `expires_at` value is included in the Name Note. The claim request's block MTP and requested registration term are policy inputs and are not repeated as transition fields.

5.5.2 Expiration

For a fixed-term registration, the expiration condition is reached when:

$$\text{canonical_chain_mtp} \geq \text{expires_at}.$$

Reaching `expires_at` does not directly modify derived registry state. When the expiration condition is reached, the Mint MUST create a `release` Name Note.

The registration remains active in derived registry state until a `release` Name Note is accepted on the canonical Zcash chain, or the registration otherwise ends under the rules in Section 7.3.

A registration whose `expires_at` value is `none` has no fixed expiration but remains subject to the liveness requirement.

5.5.3 Updates and renewals

An ordinary `update` does not require the user to provide `expires_at`. The Mint reads the current accepted registration state and carries the current `expires_at` into the successor Name Note.

An ordinary `update` MUST NOT extend, shorten, restart, or remove the current registration period.

An `update` MAY request a registration extension. The user supplies the requested additional term, not an absolute expiration timestamp.

For a finite registration:

$$\text{new_expires_at} = \text{current_expires_at} + \text{requested_term}.$$

If no extension is requested:

$$\text{new_expires_at} = \text{current_expires_at}.$$

A registration without fixed expiration (`expires_at` of `none`) MUST NOT be extended: an update requesting a term against such a registration is refused.

The extension is added to the existing `expires_at`, rather than to the current time. The resulting `expires_at` MUST NOT be more than 99 years ahead of the canonical-chain MTP at which the Mint authorizes the renewal.

An `update` with the term `forever` requests an upgrade of a fixed-term registration to a registration without fixed expiration; an upgrade is not an ordinary `update`. The resulting Name Note encodes `expires_at` as `none`; the registration's liveness obligations are unchanged. The upgrade is subject to the price and payment requirements of the active pricing policy, in addition to the request fee; its payment is the Respond, as for any `update`. An upgrade MUST NOT be requested against a registration without fixed expiration.

Once the expiration condition has been reached, the Mint MUST NOT accept a new renewal request and MUST create the required `release` Name Note.

An `update` and a `release` for the same registration both consume the current Name Note, so at most one of them can be accepted. If the `release` is accepted first, the registration has ended and a later `update` does not affect registry state.

An `update` MAY retain the currently bound Unified Address. Such an update may be used for renewal, liveness confirmation, or both.

5.5.4 Liveness

The Mint is responsible for tracking and enforcing liveness.

The current ZNS deployment MUST fix the liveness interval L in seconds.

For a registration whose most recent liveness time is τ, the liveness deadline is:

$$\text{liveness_deadline} = \tau + L.$$

The initial liveness interval begins when the `claim` Name Note is accepted on the canonical Zcash chain. A successfully authorized `update` resets the liveness interval when the resulting Name Note is accepted on the canonical chain, including an update that retains the currently bound Unified Address.

The liveness time of a `claim` or `update` is derived from the canonical-chain MTP associated with the block containing that accepted Name Note.

The Mint records the most recent successful liveness event in protected persistent state and derives the next liveness deadline using canonical-chain MTP. If a chain reorganization removes or changes a `claim` or `update` used to establish the current liveness time, the Mint MUST reconcile its protected lifecycle state with the replacement canonical chain before enforcing the resulting liveness deadline.

A Request or Relay does not satisfy liveness. A failed, expired, or incomplete Respond does not satisfy liveness. A liveness interval resets only when the resulting `update` Name Note is accepted on the canonical Zcash chain.

If the required liveness interval passes without a successful liveness event, the Mint MUST create a `release` Name Note.

For a fixed-term registration, the Mint MUST create a release when ei-

ther the committed `expires_at` is reached or the liveness requirement fails, whichever occurs first.

An `update` affects the registration only if it consumes the current Name Note before the required `release` does.

Liveness enforcement is performed by the attested Mint and is not part of the Resolver's required state-derivation procedure.

5.5.5 Release conditions

The Mint MUST create a `release` Name Note when any of the following occurs:

1. the controller successfully completes the required authorization procedure for an explicit `release`;
2. canonical-chain MTP reaches the committed `expires_at`; or
3. the current liveness deadline is reached without an accepted `update` Name Note having reset the liveness interval.

The first condition is a controller-requested release and requires the OTP authorization procedure defined in Section 6. The second and third conditions are lifecycle releases created by the Mint and do not require user authorization.

Reaching an expiration or liveness deadline does not itself modify registry state. Registry state changes only when the resulting `release` Name Note is accepted on the canonical Zcash chain, or when another transaction ends the registration under the rules in Section 7.3.

An `update` affects the current registration only if it consumes the current Name Note before the `release` does.

After a `release`, a subsequent `claim` begins a new registration and uses the zero predecessor defined in Section 4.

5.6 On-chain completeness

A Resolver must be able to reconstruct registry state from the on-chain Name Note history without relying on the Mint as a data source. Every accepted transition must therefore be fully represented in its Name Note. The Mint must not omit a transition field or replace it with an off-chain pointer.

For a `claim` or `update`, the resulting `expires_at` is part of the transition

and MUST be present in the Name Note. The requested registration term, requested extension, and MTP values used by the Mint to apply lifecycle policy are policy inputs and need not be repeated in the Name Note.

For a `release`, the Name Note records the Unified Address of the binding being released.

The protocol's validity limits on names, Unified Addresses, and other encoded fields MUST ensure that every valid ZNS transition fits within a single 512-byte Name Note memo.

6 User Authorization

Zcash shielded addresses do not provide a standard message-signing mechanism for authorizing changes to an existing name. ZNS therefore uses an in-band one-time passcode (OTP) delivered through Zcash's native shielded encryption.

A `claim` has no existing controller and therefore requires no OTP.

A lifecycle `release` created by the Mint to enforce expiration or liveness does not require an OTP.

6.1 Request, Relay, and Respond

An `update` or controller-requested `release` is three memos: Request, Relay, and Respond.

Request. The controller submits the Request specified in Section 3 as an Ironwood output to the Mint's treasury account. That note MUST carry the request fee of one US dollar, converted at the rate of the active pricing policy and rounded up to the next 100,000-zatoshi step. The fee provides Sybil resistance: an underpaid Request is rejected and its value is retained by the Mint. The fee is not the name price. The Relay is funded from the Mint's treasury account.

Relay. The Mint sends the Relay to the Ironwood receiver contained in the Unified Address currently bound to the name. The Mint MUST reject the exchange if that Unified Address does not contain an Ironwood receiver. Successful authorization establishes access to that receiver only, not to any other receiver contained in the Unified Address.

Respond. The controller returns the memo it received, unchanged, to the same treasury account:

```
ZNS:otp:<otp>:<name>:<verb>:<ua>
```

The Relay and the Respond differ only in sender.

The name payment is on the Respond. That note MUST satisfy the price and payment requirements of the active pricing policy. The Request funds only the Relay; it is not the name price.

The OTP is bound to the requested transition, expires after a fixed time interval measured using canonical-chain MTP, and is consumed on successful use. Receipt of the correct OTP within that window demonstrates both the ability to decrypt shielded messages sent to the selected receiver and temporal liveness.

For an update, authorization does not verify control of the proposed new Unified Address. A valid but unintended or substituted Unified Address can therefore become the new binding.

An update MAY specify the currently bound Unified Address as the target address when performed for renewal, liveness confirmation, or both.

6.2 Relay memo

The Relay is an Ironwood shielded memo from the Mint to the controller. The non-padding bytes are:

```
ZNS:otp:<otp>:<name>:<verb>:<ua>
```

The six-digit code is the first field after the message type so a wallet displays it without the controller scanning later fields. The `<otp>` field is exactly six ASCII decimal digits, including leading zeroes. The `<name>` field is the name being updated or released. The `<verb>` field is `update` or `release`. The `<ua>` field is the target Unified Address from the Request.

The memo is exactly 512 bytes and zero-padded. The `otp` message type distinguishes it from a Request. A Request parser MUST reject it; the Mint's parser accepts it as a Respond.

Relay memos are sent from the Mint treasury account. That account's viewing key is not the published Name Note viewing key.

6.3 OTP validity

The Mint generates each OTP uniformly from `000000` through `999999` using a cryptographically secure random source inside the attested environment [8].

For an `update`, the Mint computes the resulting `expires_at` before issuing the OTP and binds the OTP to the exact `(name, action, ua, expires_at)` tuple in its protected state.

For an ordinary update, `expires_at` is the current accepted expiration value carried forward unchanged. For a renewal, `expires_at` is the value obtained by adding the requested additional term to the current accepted expiration.

The user is not required to supply the resulting absolute `expires_at` as a request field. The Mint MUST accept the OTP only for the same pending update and MUST reject it if the resulting `(name, action, ua, expires_at)` would differ from the tuple to which the OTP was issued.

For a controller-requested `release`, the OTP remains bound to the applicable `(name, action, ua)` values. Lifecycle releases created by the Mint to enforce expiration or liveness do not use OTP authorization.

The current ZNS deployment MUST fix the OTP validity duration D_{OTP} in seconds and the maximum number of verification attempts.

If an OTP is issued when canonical-chain MTP is τ, define:

$$\mathrm{otp_expires_at} = \tau + D_{\mathrm{OTP}}.$$

The Mint MUST reject the OTP when canonical-chain MTP is greater than or equal to `otp_expires_at`.

The Mint MUST also reject an OTP presented for a different bound transition, an OTP that exceeds the permitted number of attempts, or an OTP that has already been successfully used. A successfully used OTP is consumed and MUST NOT be accepted again for the same Request. An incorrect OTP does not consume the challenge.

If the accepted Name Note for the name changes, including under a chain reorganization, the Mint MUST discard the pending OTP.

OTP expiration is independent of registration expiration and the liveness deadline.

6.4 Public verification boundary

Request, Relay, and Respond are not part of the public Name Note history. The Resolver derives registry state from Name Notes. It can check the resulting note, including the `expires_at` committed by an `update`, and Mint authorization and attestation. It cannot recover the OTP, or prove from those notes that an OTP was issued for that transition.

Under the TEE assumptions, attestation is what establishes that the Mint ran the approved authorization procedure.

7 The Resolver

The Zcash chain records Name Notes in canonical order, but it does not determine which transitions belong in the registry. A Resolver verifies each Name Note and applies accepted transitions to derive registry state.

Anyone can independently run a Resolver. Resolvers reading the same canonical Zcash chain under the current protocol rules must derive the same registry state.

7.1 Resolver responsibilities

A Resolver must:

1. scan the canonical Zcash chain using the Mint's published full viewing key to identify and decrypt Ironwood outputs, obtaining their memos and note components;
2. parse the decrypted outputs to identify Name Notes;
3. verify each Name Note's commitment using the derivation of its `rseed`, ψ, and `rcm` defined in Section 4;
4. verify Mint authorization and attestation as specified in Section 5;
5. apply accepted transitions in canonical order to derive registry state; and
6. provide clients with the current and historical binding state of each name, including the `expires_at` value committed by the applicable Name Note.

If a chain reorganization changes the canonical Zcash history, the Resolver MUST roll back any state derived from removed blocks and replay the replacement chain from the common ancestor.

7.2 Verification

Before state derivation, the Resolver verifies two properties of each Name Note:

1. Commitment verification. The Resolver parses the encoded transition and reconstructs the note commitment as specified in Section 4. The Name Note passes commitment verification only if the reconstructed note commitment matches the `cmx` recorded on-chain.

2. Mint authorization. The Resolver uses the Mint's published viewing key to detect Name Note outputs. Detection alone does not establish that the Mint created an output: the viewing key is public, so any party can construct a well-formed Name Note output to the Name Note account. A Name Note is Mint-authorized only if its transaction spends a note that only the Mint spending key can spend: the current accepted Name Note of the name for an `update` or `release`, or a live anchor for a `claim`, as specified in Section 7.3. The Resolver also verifies the Mint's attestation as specified in Section 5.

User authorization is not independently reconstructed by the Resolver. As described in Section 6, Request, Relay, and Respond are not part of the public Name Note history. Under the TEE assumptions, the Resolver relies on Mint attestation to establish that the approved authorization procedure was executed.

7.3 State derivation

The Resolver processes transactions in canonical order: ascending block height, then transaction order within the block. A transaction carrying more than one Name Note produces no accepted transition.

For each name, the `prev` field links a transition to the preceding Name Note within the current registration. A `claim`, including one following a `release`, uses the zero predecessor defined in Section 4. An `update` or `release` is accepted only if its transaction spends the current accepted Name Note of that name and no other name's, and its `prev` matches the `cmx` of that Name Note.

A `claim` has no predecessor Name Note to spend and instead spends an anchor: a zero-value note held by the Name Note account. The set of live anchors, the anchor pool, is established once during Mint setup with a fixed number of anchors. After the pool is established, other zero-value outputs to the Name Note account do not become anchors. A `claim` is accepted

only if its transaction spends exactly one live anchor, spends no live Name Note, and creates exactly one zero-value output to the Name Note account, and the name has no live registration.

Every anchor a transaction spends leaves the pool, whether or not any transition is accepted. The zero-value output created by that transaction joins the pool as a successor anchor only if the transaction spent exactly one live anchor, carries exactly one Name Note, and creates exactly one zero-value output to the Name Note account. The Resolver derives the anchor pool from the canonical chain in the same order as registry state.

Because a Name Note can be spent only once, at most one transition can consume a given Name Note. Competing transitions that reference the same predecessor cannot all be accepted; the first to consume it on the canonical chain is the only one that can affect registry state.

A transaction that spends the current accepted Name Note of a name without creating a valid successor for that name ends the registration at that transaction. No `release` Name Note is created in this case.

Each accepted transition updates the registry state for that name. An `update` or `release` whose transaction does not spend the current accepted Name Note, and a `claim` whose transaction does not spend a live anchor, do not affect the registry.

8 Trust Assumptions

ZNS relies on the following trust assumptions:

8.1 Zcash consensus

ZNS assumes that Zcash consensus provides the canonical ordering and availability of Name Notes. Registry history can be reconstructed from that chain without relying on the Mint to preserve a separate database.

ZNS treats Median Time Past derived from the canonical Zcash chain as the authoritative time source for Mint-enforced protocol periods.

8.2 TEE security

ZNS assumes that the selected TEE generates the Mint spending key, correctly executes the approved Mint code and pricing policy, and provides

trustworthy remote attestation. This includes enforcement of claim eligibility, authorization, lifecycle, and pricing rules, correct calculation of `expires_at`, and correct use of canonical-chain MTP for expiration, liveness, and OTP validity.

A compromise of these properties could allow the Mint policy to be bypassed or unauthorized Name Notes to be created, but it would not allow previously confirmed Name Notes to be removed or rewritten outside Zcash consensus.

8.3 User viewing access

ZNS assumes that the current controller retains viewing access to the Ironwood receiver of the bound Unified Address. This access is required to complete `update` and `release` authorization and to satisfy the liveness requirement. ZNS does not require or verify spending authority over that receiver.

Loss of viewing access prevents the controller from authorizing subsequent changes or satisfying the liveness requirement.

8.4 Price data

If registration pricing depends on external market data, ZNS assumes that the price data supplied to the Mint is available and accurate enough to apply the pricing policy. Incorrect price data can affect registration prices but cannot alter previously recorded Name Notes.

8.5 Protected-name access

Access codes for protected names are derived inside the attested Mint environment from a Mint-held secret and the claim name. ZNS assumes that this secret remains unavailable outside that environment, that codes distributed to authorized claimants are not disclosed to unauthorized parties, and that the Mint's read of the protected-name policy is accurate enough to classify names as open or protected. Compromise of the secret, disclosure of a valid code, or a false open classification can allow a protected name to be claimed by an unauthorized party. Once a name's protection lifts, these assumptions no longer apply to claims for that name.

8.6 Mint availability

ZNS assumes that the Mint remains available to evaluate requests and create required lifecycle `release` Name Notes. If the Mint becomes unavailable, existing registry state remains reconstructible, but new transitions cannot occur. An expiration or liveness deadline may be reached while the Mint is unavailable, but reaching the deadline does not itself change registry state. The registration remains in the derived registry until the required `release` Name Note is accepted on the canonical Zcash chain.

9 Privacy Considerations

ZNS does not provide registration privacy, resolution-query privacy, or protection against a Resolver presenting a client with stale chain state.

The Name Note account's published full viewing key allows anyone to read the ZNS registry history, including names, `claim`, `update`, and `release` transitions, associated Unified Addresses, committed `expires_at` values when present, and predecessor references. ZNS therefore does not hide name bindings or their history.

Information outside the Name Note history may also be observable. The Mint can observe Request, Relay, and Respond memos. A remotely queried Resolver can observe the names requested by a client. Wallet providers and network observers may observe connection metadata such as IP addresses and request timing, depending on how the client accesses ZNS.

Publishing a Unified Address as a name binding does not expose the shielded payments sent to that address or create a public transaction history for those payments.

10 ZNS Improvement Proposals

Protocol changes are specified through ZNS Improvement Proposals (ZNSIPs). Each ZNSIP is identified as `ZNSIP-NNN: <Title>` and stored as `znsip-NNN.md`, where `NNN` is assigned by the ZNSIP editors. A ZNSIP is submitted to the ZNSIP repository as a pull request.

Following review and acceptance by the editors, the pull request is merged into the repository. Repository inclusion records the accepted proposal but does not constitute protocol activation.

The ZNSIP repository MUST maintain a public list of all active editors.

11 Glossary

Terms defined in the body are collected here for reference. Their normative definitions remain in the referenced sections.

Term	Definition
ZNS (Zcash Name Service)	The name registry protocol specified by this document.
Name Note	An Ironwood output whose memo records a name transition, including its `expires_at` value, and whose `rseed`, and through it the note commitment, is derived from that transition; see Section 4.
Request	A user-to-Mint memo asking the Mint to evaluate a `claim`, `update`, or `release`; it never carries an OTP. A `claim` MAY carry an access code; see Section 3.
Relay	A Mint-to-user memo of the form `ZNS:otp:<otp>:<name>:<verb>:<ua>` that delivers the OTP; see Section 6.
Respond	A user-to-Mint memo that returns the OTP, `ZNS:otp:<otp>:<name>:<verb>:<ua>`; the Relay and the Respond differ only in sender; see Section 3 and Section 6.
Access code	An optional leading field of exactly six ASCII decimal digits (`0`–`9`, including leading zeroes) on a `claim` Request, used only to claim a name while it is protected; see Section 3 and Section 5. Distinct from the OTP.
Transition	A `claim`, `update`, or `release` recorded in a Name Note and represented by the tuple σ; see Section 4.
Predecessor (`prev`)	The `cmx` of the previously accepted Name Note for the same name, encoded as 64 lowercase hexadecimal characters. A `claim` uses 64 zeroes; see Section 4.
Binding	The association between a name and a Unified Address established by its accepted Name Note history.
Registry state	The set of name bindings derived from accepted Name Notes in canonical chain order; see Section 7.
Name Note viewing key	The published full viewing key for the Mint account used to create Name Notes.
Mint	The protocol's single registrar. It evaluates name requests under the naming policy and creates Name Notes for accepted requests; see Section 5.

Term	Definition
Mint authorization	Authorization provided by the recognized Mint spending key, evidenced on-chain by the spend of a Mint-held note (the predecessor Name Note, or an anchor for a `claim`), and accepted subject to the attestation requirements in Section 5; see Section 7.
Anchor	A zero-value note held by the Name Note account whose spend authorizes a `claim`. Each accepted `claim` spends one anchor and creates its successor; see Section 7.3.
Resolver	A client that scans the canonical Zcash chain using the Name Note viewing key, verifies Name Notes, derives registry state according to Section 7, and provides current and historical binding state, including committed expiration values.
User authorization	Request, Relay, and Respond, used to authorize an `update` or controller-requested `release`. For an `update`, the OTP is bound to `(name, action, ua, expires_at)`; see Section 6.
One-time passcode (OTP)	A six-digit decimal passcode delivered in the Relay and returned in the Respond; see Section 6.
Expiration (`expires_at`)	The absolute Unix timestamp in whole seconds committed by a `claim` or `update` Name Note, or the exact value `none` for a registration without fixed expiration.
Registration term	A duration requested for an initial fixed-term registration or renewal, or the exact value `forever` requested to upgrade a fixed-term registration. The user supplies a term; the Mint computes the resulting absolute `expires_at`, or `none` for an upgrade; see Section 5.5.
Median Time Past (MTP)	The chain-derived time value obtained from the canonical Zcash chain and used by the Mint for protocol-defined time checks.
Liveness time	The canonical-chain MTP associated with the block containing the accepted `claim` or most recent accepted `update` for a registration.
Liveness deadline	The current liveness time plus the protocol-defined liveness interval. Reaching the deadline requires the Mint to create a `release` Name Note.
ZNSIP	A ZNS Improvement Proposal specifying a proposed protocol change; see Section 10.

*External terms inherited from the Zcash protocol specification [1] or its ZIPs are not redefined here: Ironwood action, Ironwood memo, note commitment (`cmx`), Unified Address, and incoming viewing key retain the meaning fixed by their normative sources [3, 5, 1].

References

[1] Electric Coin Company and Zcash Foundation. *Zcash Protocol Specification*. https://zips.z.cash/protocol/protocol.pdf.

[2] Zcash Improvement Proposal 212. *Allow Recipient to Derive Ephemeral Secret from Note Plaintext*. https://zips.z.cash/zip-0212.

[3] Zcash Improvement Proposal 224. *Orchard Shielded Protocol*. https://zips.z.cash/zip-0224.

[4] Zcash Improvement Proposal 2005. *Ironwood Quantum Recoverability*. https://zips.z.cash/zip-2005.

[5] Zcash Improvement Proposal 316. *Unified Addresses and Unified Viewing Keys*. https://zips.z.cash/zip-0316.

[6] S. Bradner and J. Leiba. *Key words for use in RFCs to Indicate Requirement Levels*. BCP 14, RFC 2119 and RFC 8174. https://www.rfc-editor.org/info/bcp14.

[7] Zcash Improvement Proposal 0. *ZIP Process*. https://zips.z.cash/zip-0000.

[8] zcashme. *zns-mint reference implementation*. https://github.com/zcashme/zns-mint.